IMPROVING NETWORK SECURITY THROUGH INTRUSION DETECTION SYSTEMS

Realizable Adversarial Evasion and a Robust Machine-Learning IDS

21CSC302J - Computer Networks · Case Study, Topic 10

Submitted by

[STUDENT NAME 1] [RA23XXXXXXXXXX]
[STUDENT NAME 2] [RA23XXXXXXXXXX]
[STUDENT NAME 3] [RA23XXXXXXXXXX]

Under the Guidance of

[GUIDE NAME]

[Designation], [Department of ...]

In partial satisfaction of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING

[with specialisation in ...]



SCHOOL OF COMPUTING
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603203

OCTOBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603203

BONAFIDE CERTIFICATE

Certified that this Course Project Report titled "Improving Network Security Through Intrusion Detection Systems: Realizable Adversarial Evasion and a Robust Machine-Learning IDS" is the bonafide work done by [STUDENT NAME 1] [RA23XXXXXXXXXXX], [STUDENT NAME 2] [RA23XXXXXXXXXXX], [STUDENT NAME 3] [RA23XXXXXXXXXXX], who carried it out under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other work.

[GUIDE NAME]
[Designation]
[Department of ...]
SRM Institute of Science and Technology
Kattankulathur Campus, Chennai

Table of Contents

Abstract 5
1. Introduction 6
1.1 Intrusion Detection in Computer Networks 6
1.2 Machine Learning for Intrusion Detection 6
1.3 Adversarial Attacks on ML-based IDS 6
1.4 Problem Statement 6
1.5 Objectives 6
1.6 Contributions 7
1.7 Organisation of the Report 7
2. Requirement Analysis 8
2.1 Functional Requirements 8
2.2 Threat Model 8
2.3 Hardware Requirements 8
2.4 Software Requirements 9
3. Architecture & Design 10
3.1 Placement of the IDS in the Network 10
3.2 System Architecture 10
3.3 Network Design of the Capture Lab 10
3.4 Network Configuration Table 11
3.5 Connection Features from Packets 11
4. Dataset and Feature Analysis 13
4.1 The NSL-KDD Dataset 13
4.2 The 41 Features 13
4.3 Preprocessing 13
4.4 Feature Realizability Taxonomy 13
5. Methodology and Implementation 15
5.1 Baseline IDS (the Paper's Model) 15
5.2 Threat Model: Transfer Attack through a Surrogate 15
5.3 FGSM and PGD 15
5.4 The Realizability Mask 15
5.5 The Improved IDS 16
5.6 Adversarial Training and the Adaptive Attacker 16

5.7 Evaluation Metrics 16
5.8 Implementation of the Capture Lab 17
5.9 Demonstration Application 17
6. Results and Testing 18
6.1 Detection Performance 18
6.2 Robustness of the Threshold Choice 19
6.3 Accuracy under Attack: the Published Protocol 20
6.4 Evasion Rate 20
6.5 Feature Importance versus Controllability 21
6.6 Validation of the Capture Pipeline 22
6.7 Reproducibility 22
6.8 Demonstration Application 22
7. Discussion 27
7.1 Comparison with Related Work 27
7.2 Limitations 28
8. Conclusion and Future Work 29
References 30
Appendix A: Full Feature Realizability Taxonomy 31
Appendix B: Reproducing the Results 32

Abstract

An intrusion detection system (IDS) watches network traffic and raises an alarm when a connection looks malicious. Machine-learning IDSs are usually stress-tested with adversarial examples that nudge every input feature freely until the detector is fooled. On a real network, however, an attacker controls only the packets they send: most of the 41 NSL-KDD connection features are decided by the victim host or by other users' traffic, and cannot be set by the attacker at all.

This case study first reproduces a published pipeline (a Random Forest IDS on NSL-KDD, attacked with FGSM and PGD and defended by adversarial training) and then extends it in three ways. First, every feature is classified by who controls it over TCP/IP, and the result is enforced as a **realizability mask** that constrains attacks in real network units; only 9 of 41 features remain movable. Second, we show that the usual "accuracy under attack" metric mixes genuine evasion with false alarms manufactured on benign traffic, and measure the **evasion rate** instead. Third, we build an **improved IDS**: a soft-voting ensemble whose alarm threshold is calibrated on attack types held out of training.

On the KDDTest+ set, which contains 17 attack types never seen in training, the improved IDS raises accuracy from 77.9% to 89.6% and F1-score from 0.765 to 0.907. It cuts the evasion rate from 31.1% to 4.9% against an unrestricted PGD attacker, and from 13.0% to 3.0% against a realistic one. Adversarial training on unrestricted perturbations blocked every unrestricted attack yet still let 7.1% of realistic attacks through. An isolated Docker capture lab with a PCAP-to-feature extractor, and an offline demonstration application, accompany the study.

Keywords: intrusion detection, adversarial machine learning, NSL-KDD, network security, realizable attacks, evasion rate.

1. Introduction

1.1 Intrusion Detection in Computer Networks

An intrusion detection system monitors a network or a host for malicious activity and policy violations. A **network IDS (NIDS)** observes packets at a vantage point in the network, typically a switch mirror (SPAN) port or a network tap, while a **host IDS** runs on an individual machine. Signature-based NIDSs such as Snort and Suricata match traffic against rules for known attacks; they are precise but blind to anything without a rule. Anomaly-based and machine-learning (ML) NIDSs learn what malicious traffic looks like and can flag attacks they were never explicitly told about, at the cost of more false alarms [20].

1.2 Machine Learning for Intrusion Detection

ML-based detectors do not read raw packets. Each TCP/UDP/ICMP connection is first summarised as a fixed set of features: how long it lasted, how many bytes flowed each way, how it ended, and how many similar connections reached the same host recently. The NSL-KDD dataset [1], a cleaned version of the KDD Cup 1999 data [9], describes every connection with 41 such features and a label. A classifier trained on these records labels new connections as normal or attack.

1.3 Adversarial Attacks on ML-based IDS

ML models can be fooled by adversarial examples: inputs altered by small, carefully chosen amounts so that the model changes its decision. The Fast Gradient Sign Method (FGSM) [2] and Projected Gradient Descent (PGD) [3] are the standard attacks. The work this study builds on [18] trained a Random Forest IDS on NSL-KDD, explained it with SHAP [6], attacked it with FGSM and PGD, and defended it with adversarial training.

1.4 Problem Statement

Those attacks perturb **all 41 features freely**. But a feature vector is not traffic. An attacker cannot edit a number; they can only send packets, and the wire limits what those packets can change. The number of bytes the attacker sends is theirs to choose. Whether the victim answered with a reset, or what fraction of the victim's last 100 connections came from other hosts, is not. An adversarial example that no packet sequence can produce is a mathematical artefact, not a threat. This report asks how much of the published adversarial threat survives when attacks are limited to what a real attacker can actually send, and whether a better-calibrated IDS is more robust than one hardened by adversarial training.

1.5 Objectives

- Reproduce the published Random Forest IDS, its FGSM/PGD attacks and its adversarial-training defence.
- Classify all 41 NSL-KDD features by who controls them over TCP/IP, and enforce the classification as a realizability mask applied inside the attacks.

- Replace whole-test-set "accuracy under attack" with an evasion rate that measures only attacks disguised as normal traffic.
- Build an IDS with higher honest accuracy on unseen attacks, without using the test set to tune it.
- Provide an isolated capture lab and extractor that turn real packets into the same 41 features, and an offline application that demonstrates the results.

1.6 Contributions

- **A realizability mask** that enforces protocol constraints in raw network units (whole bytes, raise-only connection counts, valid ranges) rather than in the model's scaled feature space.
- **A measurement showing that accuracy under attack is misleading:** most of the measured drop comes from benign traffic made to look malicious, which an attacker does not control.
- **Evidence that a calibrated threshold outperforms adversarial training** against realistic attackers, while also raising accuracy on unseen attacks.
- **An end-to-end capture pipeline and demonstration application** that tie the ML result back to real network traffic.

Constrained adversarial attacks on NIDSs have been studied before [11]–[16]; Section 7.1 positions this work against that literature.

1.7 Organisation of the Report

Section 2 lists the requirements and the threat model. Section 3 presents the architecture and the network design of the capture lab. Section 4 describes the dataset and the feature realizability taxonomy. Section 5 covers the methodology and implementation, Section 6 the results and testing, Section 7 the discussion and limitations, and Section 8 concludes.

2. Requirement Analysis

Requirement analysis fixes what the IDS must do, what the attacker can and cannot do, and what hardware and software the study needs.

2.1 Functional Requirements

- **Detection:** classify each network connection as normal or attack.
- **Generalisation:** detect attack types that were absent from training, as KDDTest+ requires.
- **Low false-alarm rate:** keep precision high so that operators are not flooded with alerts.
- **Robustness:** resist attackers who reshape their traffic to evade detection.
- **Honest evaluation:** no test-set information may influence training or threshold selection.
- **Reproducibility:** every reported number regenerates from one command with fixed random seeds.
- **Deployability:** the demonstration runs offline on an ordinary laptop.

2.2 Threat Model

The attacker in this study is realistic rather than all-powerful:

- **Controls only their own traffic:** payload size, connection duration, how many connections they open, and crafted header fields.
- **Cannot control the victim's responses:** whether a connection completes, is reset or is ignored is decided by the victim, as is how many bytes it sends back.
- **Cannot control third-party traffic:** the host-based features summarise the last 100 connections to the victim from every source, including legitimate users.
- **Has no access to the deployed IDS:** the attacker trains their own surrogate model and transfers the attack, rather than reading the defender's model.

2.3 Hardware Requirements

Table 1: Hardware requirements

Component	Used in this study	Minimum
Processor	Intel Core i7-1165G7, 4 cores / 8 threads	Any 64-bit x86 CPU
Memory	16 GB	8 GB
Graphics	Intel Iris Xe (integrated); no GPU used	Not required
Storage	~0.5 GB Python environment, 21.5 MB dataset	2 GB free
Capture lab host	Separate machine running Docker	Docker Engine or Desktop, 4 GB RAM
Network	Lab network is internal (no Internet)	Internet only for first-time setup

2.4 Software Requirements

Table 2: Software requirements

Software	Version	Purpose
Operating system	Windows 11 (analysis), Linux containers (lab)	Host and lab platform
Python	3.12.13	Implementation language
scikit-learn	1.9.0	Random Forest, Extra Trees, gradient boosting, metrics
Adversarial Robustness Toolbox	1.20.1	Adversarial-ML toolkit [5]
SHAP	0.52.0	Feature-importance analysis [6]
NumPy / pandas / SciPy	2.5.3 / 3.0.5 / 1.18.1	Numerical work and data handling
Matplotlib	3.11.1	Figures
dpkt	1.9.8	Reading PCAP files in the extractor
Docker	Engine/Desktop 29.x	Isolated capture lab
tcpdump, nmap, hping3	Debian/netshoot packages	Capture and traffic generation
Web browser	Chrome or Edge	Demonstration application

3. Architecture & Design

3.1 Placement of the IDS in the Network

A network IDS must see traffic without slowing it down. It is therefore attached to a switch **mirror (SPAN) port** or an inline **network tap**, which copies every frame to the sensor while the original continues to its destination. The sensor's interface runs in **promiscuous mode** so that it accepts frames addressed to other hosts. Because the sensor is passive, a failure or overload of the IDS cannot interrupt the network. It also sees **both directions** of each connection, which is essential: features such as the connection flag and the bytes returned by the server can only be computed from the server's replies.

3.2 System Architecture

Fig 1 shows the two halves of the system. The defender path takes NSL-KDD records through preprocessing into the IDS models and evaluates them. The attacker path trains a surrogate model on the same feature space, generates FGSM/PGD perturbations, passes them through the realizability mask after every step, and sends the result to the real IDS to see whether it is still detected.

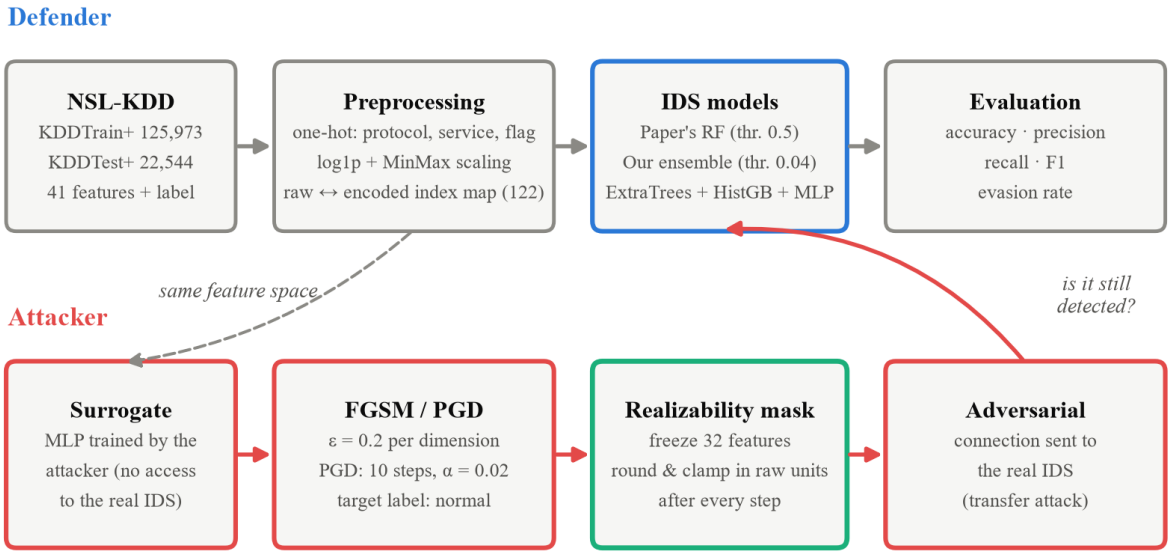


Fig 1: System architecture: defender pipeline and attacker pipeline

3.3 Network Design of the Capture Lab

To connect the ML results to real traffic, the study includes an isolated capture lab built with Docker Compose (Fig 2). Four containers share one bridge network that is declared **internal**, so it has no gateway and no route to the campus LAN or the Internet; nothing in the lab can reach a network the authors do not own.

- **Victim server:** a multi-port TCP service. Open ports complete the handshake (flag SF); closed ports answer a SYN with a reset (flag REJ).

- **Attacker:** generates reconnaissance sweeps and scripted probes that try to hit target feature values.
- **Third-party host:** sends ordinary HTTP traffic. It exists to show that the victim's host-based features change even when the attacker's own traffic is unchanged.
- **IDS sensor:** tcpdump running inside the victim's network namespace, which gives the same vantage point as a SPAN port in front of the victim.

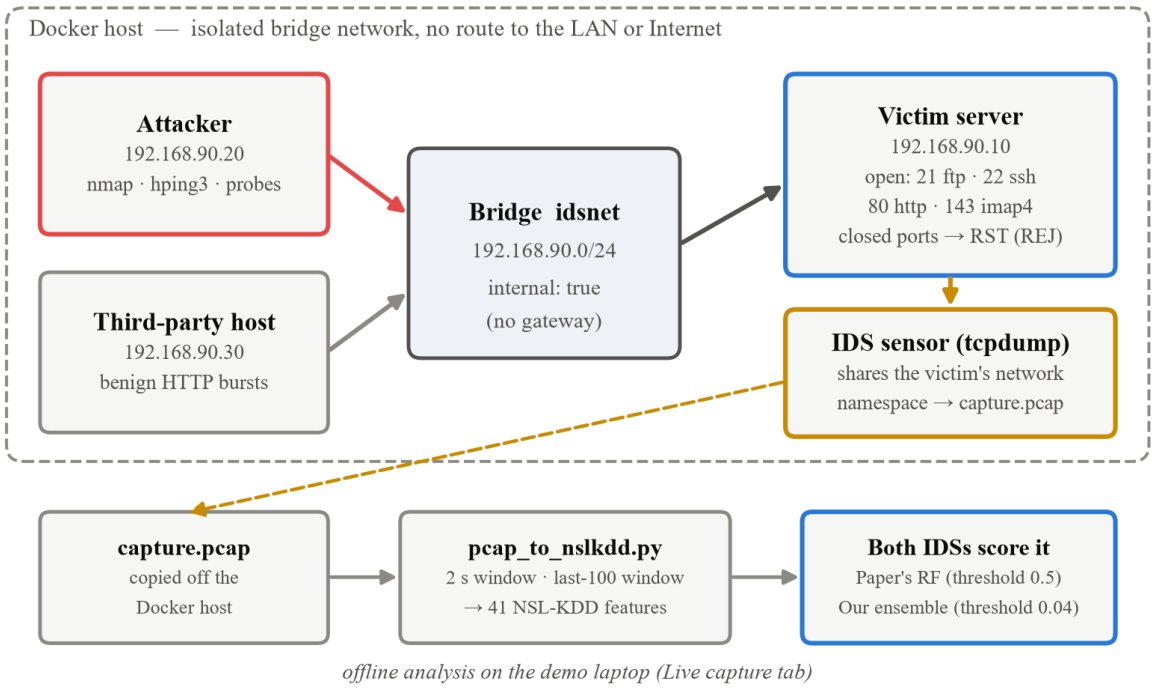


Fig 2: Network topology of the isolated capture lab

3.4 Network Configuration Table

Table 3: Network configuration of the capture lab

Device	Role	IP address	Subnet mask	Gateway	Ports / services
Bridge idsnet	Isolated segment	192.168.90.0/24	255.255.255.0	None (internal)	-
Victim server	Target host	192.168.90.10	255.255.255.0	None	TCP 21, 22, 80, 143 open; other ports reset
Attacker	Attack traffic	192.168.90.20	255.255.255.0	None	nmap, hping3, probes
Third-party host	Benign traffic	192.168.90.30	255.255.255.0	None	HTTP client
IDS sensor	Packet capture	Victim's namespace	-	-	tcpdump, 5 × 50 MB ring buffer
Demo application	Analysis and UI	127.0.0.1	-	-	HTTP on TCP 8765

3.5 Connection Features from Packets

The extractor groups packets into TCP connections and recovers each NSL-KDD feature the same way the original dataset defined it [9]. The connection **flag** comes from the TCP three-way handshake and teardown [17], [19], observed in both directions:

Table 4: TCP connection flags recovered by the extractor

Flag	Meaning	Observed when
SF	Normal establishment and termination	SYN, SYN-ACK, then FIN
S0	Connection attempt, no reply	SYN only
REJ	Connection rejected	SYN answered with RST
S1	Established, not closed	Handshake completed, no FIN or RST
RSTO	Originator aborted	Established, then RST from the client
RSTR	Responder aborted	Established, then RST from the server
SH	SYN then FIN, no reply	SYN and FIN from the client only
OTH	Mid-stream traffic	No SYN captured

The **time-based** features (23–31) count connections to the same host or service in the **past two seconds**. The **host-based** features (32–41) are computed over the **last 100 connections to the destination host, from any source**. That single definition is why an attacker cannot set the host-based features: they depend on what everyone else sent to the victim.

4. Dataset and Feature Analysis

4.1 The NSL-KDD Dataset

NSL-KDD [1] provides a training file and a separate test file. The test file is deliberately harder: 17 of its attack types never appear in training, so a model is scored on its ability to recognise new attacks rather than to memorise old ones.

Table 5: NSL-KDD files used in this study (binary labels)

File	Connections	Normal	Attack	Attack share	Attack types
KDDTrain+	125,973	67,343	58,630	46.5%	22
KDDTest+	22,544	9,711	12,833	56.9%	37

Attack types present only in KDDTest+: apache2, httptunnel, mailbomb, mscan, named, processtable, ps, saint, sendmail, snmpgetattack, snmpguess, sqlattack, udpstorm, worm, xlock, xsnoop, xterm.

4.2 The 41 Features

Table 6: Feature groups of NSL-KDD

Group	Features	Computed from	Examples
Basic	1–9	Packet headers of the connection	duration, service, flag, src_bytes
Content	10–22	Host state and session content	logged_in, num_failed_logins
Time-based	23–31	Connections in the past 2 seconds	count, serror_rate
Host-based	32–41	Last 100 connections to the same host	dst_host_count, dst_host_serror_rate

4.3 Preprocessing

The three text features are one-hot encoded: protocol_type into 3 columns, service into 70 and flag into 11. With the 38 numeric features this gives a 122-dimensional input. Numeric features are scaled to [0, 1] with MinMax scaling fitted on the training file only. For the improved IDS, the 23 heavy-tailed magnitude features (bytes, counts, durations) are first transformed with log(1 + x): bytes range from 0 to over a billion, and without this nearly every src_bytes value is compressed to almost zero.

An explicit **index map** records which encoded columns belong to each of the 41 raw features, and a two-way mapping converts encoded values back to raw units (bytes, seconds, counts). The realizability mask depends on both, as Section 5.4 explains.

4.4 Feature Realizability Taxonomy

Each feature is assigned a class according to whether, in a live attack over TCP/IP, the attacker can set it to an arbitrary value:

Table 7: Feature realizability classes

Class	Count	Meaning	Features
A — free	1	Attacker sets it directly	src_bytes
B — constrained	9	Attacker influences it within protocol rules: whole numbers, raise-only, binary or self-incriminating	duration, flag, land, wrong_fragment, urgent, num_failed_logins, su_attempted, count, srv_count
C — uncontrollable	31	Decided by the victim, by third-party traffic, or fixed when the attack is designed	All other features

Of the nine Class B features, flag cannot move under a small continuous step because it is a category, so it is frozen in the attacks. That leaves **9 movable features**: src_bytes and eight Class B features. The full 41-feature classification with a reason for each is in Appendix A.

5. Methodology and Implementation

5.1 Baseline IDS (the Paper's Model)

The baseline is a Random Forest [7], implemented with scikit-learn [10], with the published configuration: 100 trees, maximum depth 15, minimum samples per split 5 and random state 42, trained on KDDTrain+ with MinMax-scaled features. A connection is labelled an attack when the predicted attack probability is at least 0.5.

5.2 Threat Model: Transfer Attack through a Surrogate

FGSM and PGD need gradients: the direction in which each input should move to lower the attack score. A Random Forest provides none, and a real attacker would not have the defender's model anyway. The attacker therefore trains their **own surrogate model** on the same feature space, crafts the attack against it, and sends the result to the real IDS (a transfer attack). The surrogate is a small neural network (122 → 64 → 32 → 2, ReLU, Adam, 12 epochs) implemented with exact backpropagation in NumPy; it reaches 80.6% accuracy on KDDTest+.

A linear surrogate was tried first and rejected. Its input gradient is always ±w, so every perturbation of a given class pointed the same way and the two classes moved in exactly opposite directions. The perturbation therefore encoded the label, and a model adversarially trained on those examples scored a meaningless 100% — a known effect called label leaking [4]. The attacks also use the surrogate's **predicted** labels rather than the true ones, for the same reason.

5.3 FGSM and PGD

Let x be an encoded connection, g the surrogate, L the cross-entropy loss and y the label the attacker wants to move away from. FGSM takes one step of size ε in the sign of the gradient:

$$x' = x + \varepsilon \cdot sign(\nabla_x L(g(x), y))$$

PGD takes T smaller steps of size α and, after each one, projects back into the ε-ball around x and the valid range:

$$x^{(t+1)} = \Pi [x^{(t)} + \alpha \cdot sign(\nabla_x L(g(x^{(t)}), y))], \quad t = 0, \ldots, T - 1$$

This study uses ε = 0.2, α = 0.02 and T = 10. The published settings (α = 0.01, T = 5) let PGD move at most 0.01 × 5 = 0.05 per feature against a budget of 0.2, so it could never use its own budget and was weaker than the published FGSM. The range constraint is widened to include each clean test point, so that an unperturbed connection is never altered by the clipping alone.

5.4 The Realizability Mask

The mask projects a perturbed connection back onto what the wire allows. It runs **after every PGD step** and once after FGSM:

1. **Freeze:** all 31 Class C features and the categorical flag are reset to their original values, including every column of their one-hot blocks.
2. **Return to raw units:** the scaling (and log transform, if used) is inverted, so the next steps act on real bytes, seconds and connection counts.
3. **Round and clamp:** byte counts, durations and connection counts are rounded to whole numbers; binary features to 0 or 1; count, srv_count and num_failed_logins may only increase.
4. **Clip:** nothing may be negative, and every rate stays within [0, 1].
5. **Re-encode and re-freeze:** values are transformed back into model space and the frozen columns are reset again, so that round-trip floating-point error cannot move them.

The order matters. Rounding "0.37" in scaled space is meaningless, because 0.37 of the scaled range is not a whole number of bytes, so a mask applied in model space silently does nothing. Unit tests perturb every frozen feature heavily and assert that the model input is byte-identical afterwards; they also check whole numbers, raise-only behaviour, valid ranges, that a clean input is left unchanged, and that applying the mask twice equals applying it once. The tests run in both the plain and the log-scaled feature spaces.

5.5 The Improved IDS

The improved IDS averages the predicted attack probabilities of three models: Extra Trees [8] (300 trees), histogram-based gradient boosting (300 iterations) and the MLP described above, all on log-scaled features. The main lever, however, is the **decision threshold**. At 0.5 the paper's forest misses over a third of the attacks: unseen attack types score as suspicious, but not suspicious enough to pass 0.5.

The threshold is chosen without touching KDDTest+:

1. Hold out whole **attack types** (30% of the types in KDDTrain+) plus 20% of the normal traffic as a validation set, which reproduces the "unseen attack" condition of the test file.
2. Train on the remaining data and pick the threshold that maximises balanced accuracy on the validation set. Balanced accuracy is used because the test file's attack share differs from the training file's.
3. Repeat with five different random selections of held-out types and take the median, 0.04.
4. Retrain on all of KDDTrain+ and evaluate once on KDDTest+.

5.6 Adversarial Training and the Adaptive Attacker

The defence from the published work retrains the Random Forest on the training set plus PGD adversarial examples generated from the training set (never from the test set). A defence is only meaningful if it survives an attacker who knows it exists, so the defended model is also tested against an **adaptive attacker**, who retrains a fresh surrogate on the defended model's own predictions (agreeing with it on 93.4% of the test set) and attacks again.

5.7 Evaluation Metrics

With TP, TN, FP and FN the true/false positives and negatives (an attack is a positive):

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \qquad Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN) \qquad F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$$

The **evasion rate** measures the threat that matters for an IDS. Take the attack connections in KDDTest+ that the IDS detects before any perturbation; the attacker perturbs only those, aiming for the label "normal":

$$Evasion\ rate = |\{ x : f(x) = attack \ \ and \ \ f(x') = normal \}| \ / \ |\{ x : f(x) = attack \}|$$

Unlike accuracy under attack, it never perturbs benign traffic, which no attacker controls.

5.8 Implementation of the Capture Lab

1. Define the four services and the internal bridge network in docker-compose.yml.
2. Build the victim image (a Python multi-port TCP server) and the attacker image (Debian with nmap, hping3, curl and Python).
3. Start the lab; the sensor container begins a ring-buffered tcpdump capture in the victim's network namespace.
4. Run the scenarios: benign background traffic, a TCP connect sweep over ports 1–200, the realizability probe, and the third-party test, in which the attacker's traffic is repeated identically while only the third-party host's traffic changes.
5. Copy the PCAP to the analysis laptop and run pcap_to_nslkdd.py to obtain the 41 features per connection; score them with both IDSs.

5.9 Demonstration Application

An offline web application presents the work. It uses only the Python standard library and opens in a browser at http://127.0.0.1:8765. It has four views: an **overview** of the headline results; a **network simulation** that replays real KDDTest+ connections through an animated topology and both IDSs; an **attack lab** that runs FGSM or PGD live against either IDS, in unrestricted or realistic mode, and flags every physically impossible value; and a **live capture** view that scores a PCAP from the lab. Section 6.8 shows each view.

6. Results and Testing

All numbers in this section are produced by python src/experiment.py and python src/benchmark.py and read from the results folder when this report is built.

6.1 Detection Performance

Table 8: Detection performance on KDDTest+ (no attacker)

Model	Accuracy	Precision	Recall	F1-score
Paper's IDS (Random Forest)	77.90%	96.87%	63.22%	0.765
Adversarially trained RF	76.09%	96.57%	60.13%	0.741
Our IDS (ensemble, tuned threshold)	89.61%	92.36%	89.12%	0.907

Table 8 and Fig 3 compare the three models. The paper's Random Forest is precise (precision 96.9%) but misses 36.8% of attacks. Our IDS trades a little precision (92.4%) for far higher recall (89.1%), raising accuracy from 77.9% to 89.6% and F1 from 0.765 to 0.907. For an IDS this is the right trade: a missed attack is usually far more costly than an extra alert. Adversarial training slightly lowered the Random Forest's clean accuracy, to 76.1%.

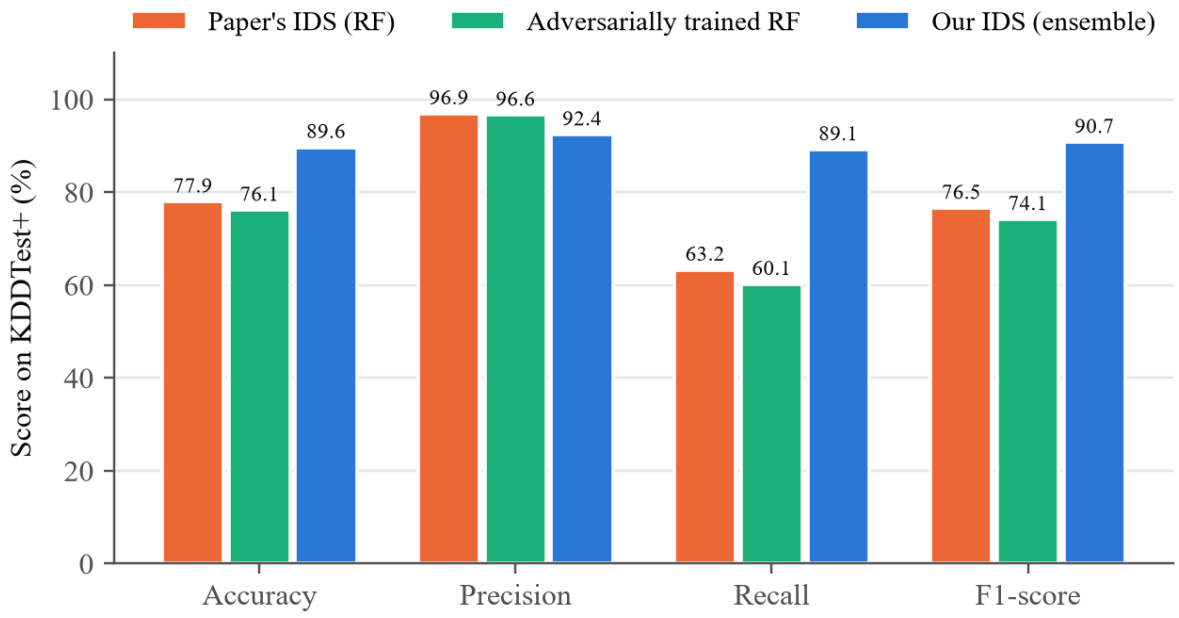


Fig 3: Accuracy, precision, recall and F1-score on KDDTest+

The confusion matrices (Fig 4) show where the gain comes from: the bottom-left cell, attacks classified as normal, shrinks sharply, at the cost of more normal connections flagged.

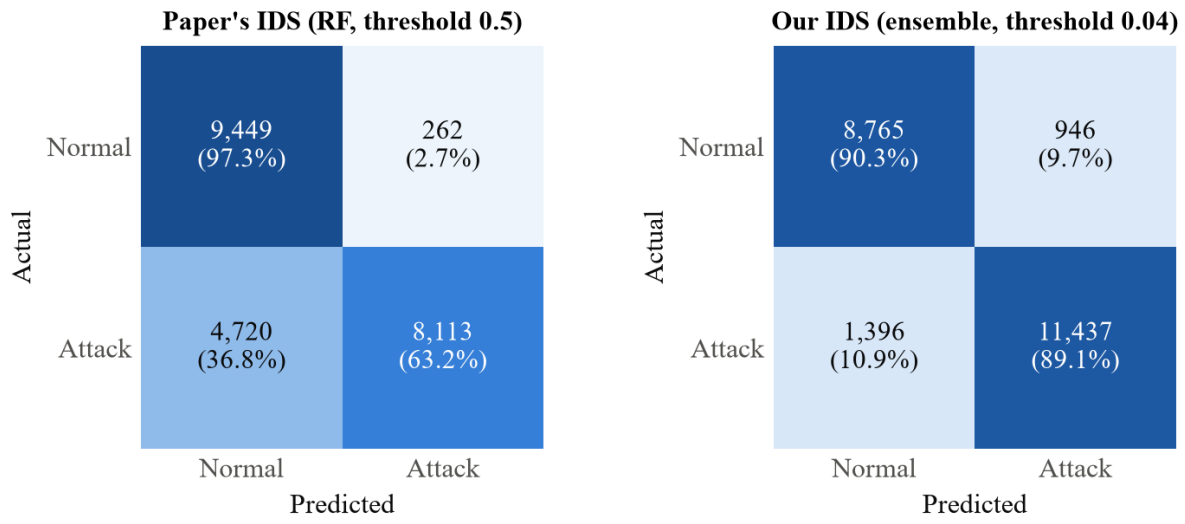


Fig 4: Confusion matrices on KDDTest+ (counts and row percentages)

6.2 Robustness of the Threshold Choice

Table 9: Accuracy before and after threshold tuning (log features; mean ± s.d. over 5 splits)

Model	Accuracy at 0.5	Accuracy, tuned (%)	Precision (%)	Recall (%)	F1
Paper's RF (100 trees, depth 15)	77.1%	88.5 ± 2.6	91.6	88.0	0.897
Random Forest (300 trees)	76.9%	89.9 ± 2.0	92.0	90.1	0.910
Extra Trees	78.9%	90.1 ± 2.2	92.2	90.3	0.912
Hist. gradient boosting	80.3%	85.5 ± 0.0	96.9	77.1	0.858
Ensemble (our IDS)	-	89.6 ± 1.0	92.3	89.1	0.907

Table 9 and Fig 5 repeat the whole select-then-test procedure over five different choices of held-out attack types. Tuning the threshold improves every model. The ensemble is the most stable (standard deviation 1.0 points), which is why it was chosen over Extra Trees, whose mean is similar but whose spread is twice as large. Histogram gradient boosting always selected the lowest threshold searched, so it shows no spread.

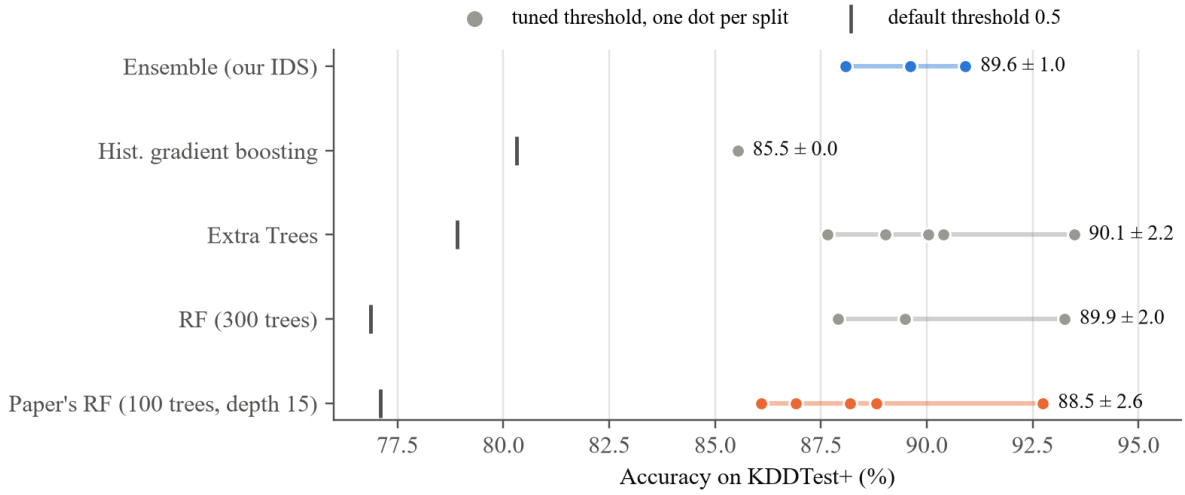


Fig 5: Test accuracy for each threshold-selection split; the bar marks the default threshold of 0.5

For contrast, the paper's Random Forest scores 99.76% on a random 80/20 split of KDDTrain+. That figure is not comparable: the test portion contains only attack types the model has already seen. It is the source of the "99%" figures common in the literature, and is not used as a result here.

6.3 Accuracy under Attack: the Published Protocol

Table 10: Whole-test-set accuracy under attack (every connection perturbed)

IDS	Attack	Accuracy	Precision	Recall	F1
Paper's IDS	No attack	77.9%	96.9%	63.2%	0.765
Paper's IDS	FGSM, unrestricted	67.7%	77.5%	60.8%	0.682
Paper's IDS	PGD, unrestricted	53.8%	58.8%	62.9%	0.608
Paper's IDS	FGSM, realistic	74.1%	95.7%	57.0%	0.715
Paper's IDS	PGD, realistic	73.7%	96.2%	56.0%	0.708
Our IDS	No attack	89.6%	92.4%	89.1%	0.907
Our IDS	FGSM, unrestricted	56.9%	56.9%	100.0%	0.725
Our IDS	PGD, unrestricted	55.3%	56.2%	97.0%	0.712
Our IDS	FGSM, realistic	73.0%	70.0%	92.0%	0.795
Our IDS	PGD, realistic	73.3%	70.5%	91.2%	0.796

Table 10 follows the published protocol, which perturbs every test connection, normal ones included. Read naively, it says our IDS is weaker: its accuracy falls from 89.6% to 55.3% under unrestricted PGD. But its **recall rises** to 97.0% while precision collapses to 56.2%. Attacks are still caught; what changed is that **normal** connections, perturbed towards "attack", now trigger alarms. The paper's model shows the same pattern: its recall barely moves (63.2% → 62.9%) while precision drops from 96.9% to 58.8%.

An attacker controls their own traffic, not other users'. The bulk of this protocol's accuracy drop therefore measures false alarms manufactured on benign traffic, not evasion, and it penalises a sensitive IDS for being sensitive. The next section measures evasion directly.

6.4 Evasion Rate

Table 11: Evasion rate: share of detected attacks disguised as normal

IDS	Attack	Attacks detected (clean)	Evasion, unrestricted	Evasion, realistic
Paper's IDS	FGSM	8,113	26.7%	13.1%
Paper's IDS	PGD	8,113	31.1%	13.0%
Adversarially trained RF	FGSM	7,717	0.0%	7.1%
Adversarially trained RF	PGD	7,717	0.0%	7.1%
Adversarially trained RF (adaptive attacker)	FGSM	7,717	0.0%	6.6%
Adversarially trained RF (adaptive attacker)	PGD	7,717	0.0%	6.9%
Our IDS	FGSM	11,437	0.1%	1.8%
Our IDS	PGD	11,437	4.9%	3.0%

Table 11 and Fig 6 report the metric that matters. Three findings stand out:

- **Our IDS is both more accurate and far harder to evade.** Against unrestricted PGD, 31.1% of the attacks the paper's IDS detects can be disguised; against ours, 4.9%.
- **Realistic attacks are much weaker.** Under the realizability mask the paper's IDS is evaded 13.0% of the time, less than half the unrestricted figure; ours, 3.0%.
- **Adversarial training defends against the wrong threat.** The defended Random Forest blocks every unrestricted attack (0.0%) yet lets 7.1% of realistic ones through (6.9% against the adaptive attacker). It learned to reject perturbations that cannot occur, not the ones that can.

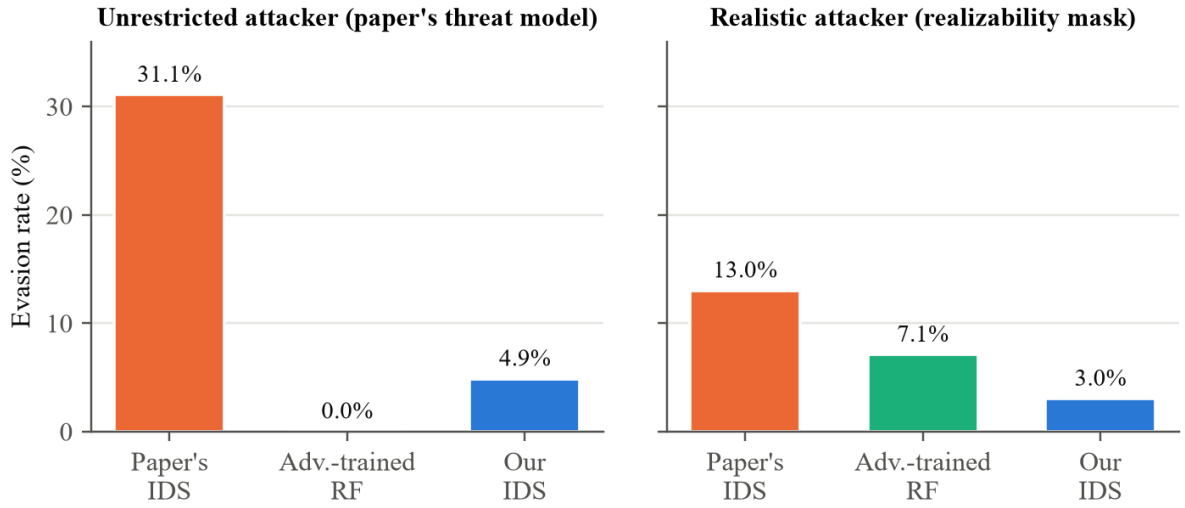


Fig 6: Evasion rate under PGD, unrestricted and realistic attackers

6.5 Feature Importance versus Controllability

Table 12: Where the paper's Random Forest places its decision weight

Feature class	Number of features	Share of model importance (SHAP)
A — free	1	1.9%
B — constrained	9	17.5%
C — uncontrollable	31	80.7%

SHAP [6] values for the paper's Random Forest explain why realistic attacks achieve so little (Table 12). 80.7% of the model's importance lies in Class C features, and only 8.5% in features an attacker can move. The five most important features are service, flag, logged_in, dst_host_srv_count, dst_host_same_srv_rate; none of them is attacker-controllable. The ranking is stable across disjoint subsamples (Spearman $\rho = 0.9995$, top-10 overlap 10/10). SHAP is used here only as supporting evidence for the realizability argument.

6.6 Validation of the Capture Pipeline

Table 13: Extractor unit tests (re-run when this report is built)

Unit test	What it checks	Result
test_flags_are_recovered	SF, REJ and S0 flags recovered from hand-built handshakes	Pass
test_src_bytes_and_service	Payload bytes in each direction and service name	Pass
test_land_flag	land = 1 when source and destination match	Pass
test_count_and_srv_count_climb	2-second window counts a burst of 40 connections	Pass
test_host_window_caps_at_100	Host window is capped at the last 100 connections	Pass
test_third_party_moves_dst_host_features	Third-party connections raise dst_host_count for the attacker	Pass

The extractor is validated against packets with known ground truth (Table 13; 6 passed, 0 failed). The last test is the third-party argument in miniature: connections the attacker never sent change the attacker's own dst_host_count.

An end-to-end check pushes a synthetic capture through the real extractor and both IDSs: 8 benign HTTP sessions from a user and a 40-port sweep from the attacker, 48 connections in all. Both IDSs flag 40/40 (paper) and 40/40 (ours) of the sweep connections; on the benign sessions the paper's IDS raises 0 alarm(s) and ours 1, consistent with the precision trade-off in Section 6.1. **The live capture on real Docker traffic has not yet been run**: the campus network blocks the plain DNS that Docker needs to download base images, so the lab is to be run on a separate machine, and its results will be added to this section.

6.7 Reproducibility

All random seeds are fixed at 42. Library versions are recorded with every run (results/run_metadata.json). The full test suite (preprocessing, mask and extractor) passes, and every figure and table in this report is regenerated from the results folder.

6.8 Demonstration Application

The figures below are screenshots of the demonstration application (Section 5.9), taken automatically from the running application.

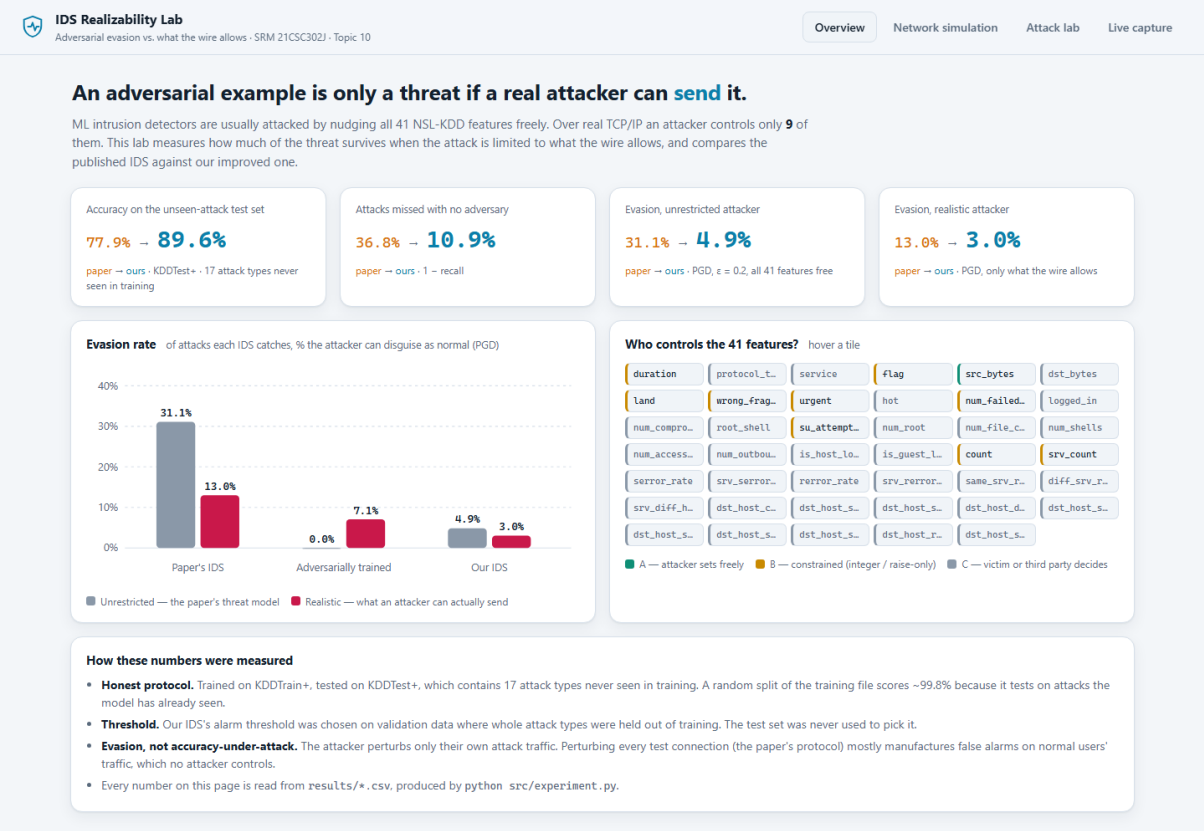


Fig 7: Overview view: headline results and who controls each feature

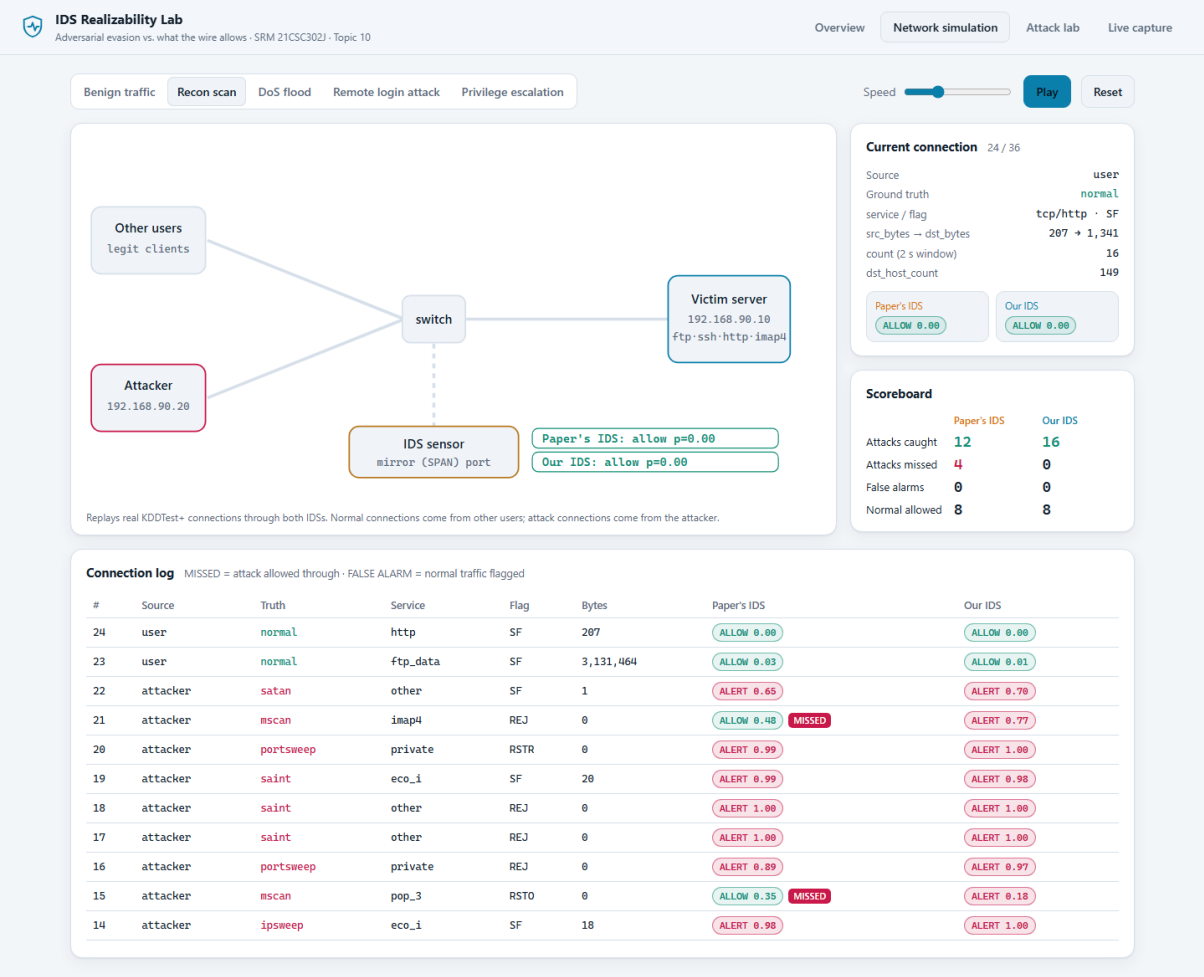


Fig 8: Network simulation: a reconnaissance scan replayed through both IDSs; the paper's IDS misses the unseen mscan attacks

Fig 9 shows an unrestricted PGD attack against the paper's IDS on an apache2 connection, an attack type absent from training. The IDS is evaded, but the attack changes 29 features and 28 of them take values no packet can carry (outlined in red): fractional durations, a protocol that is partly TCP and partly something else, a connection count that went down.

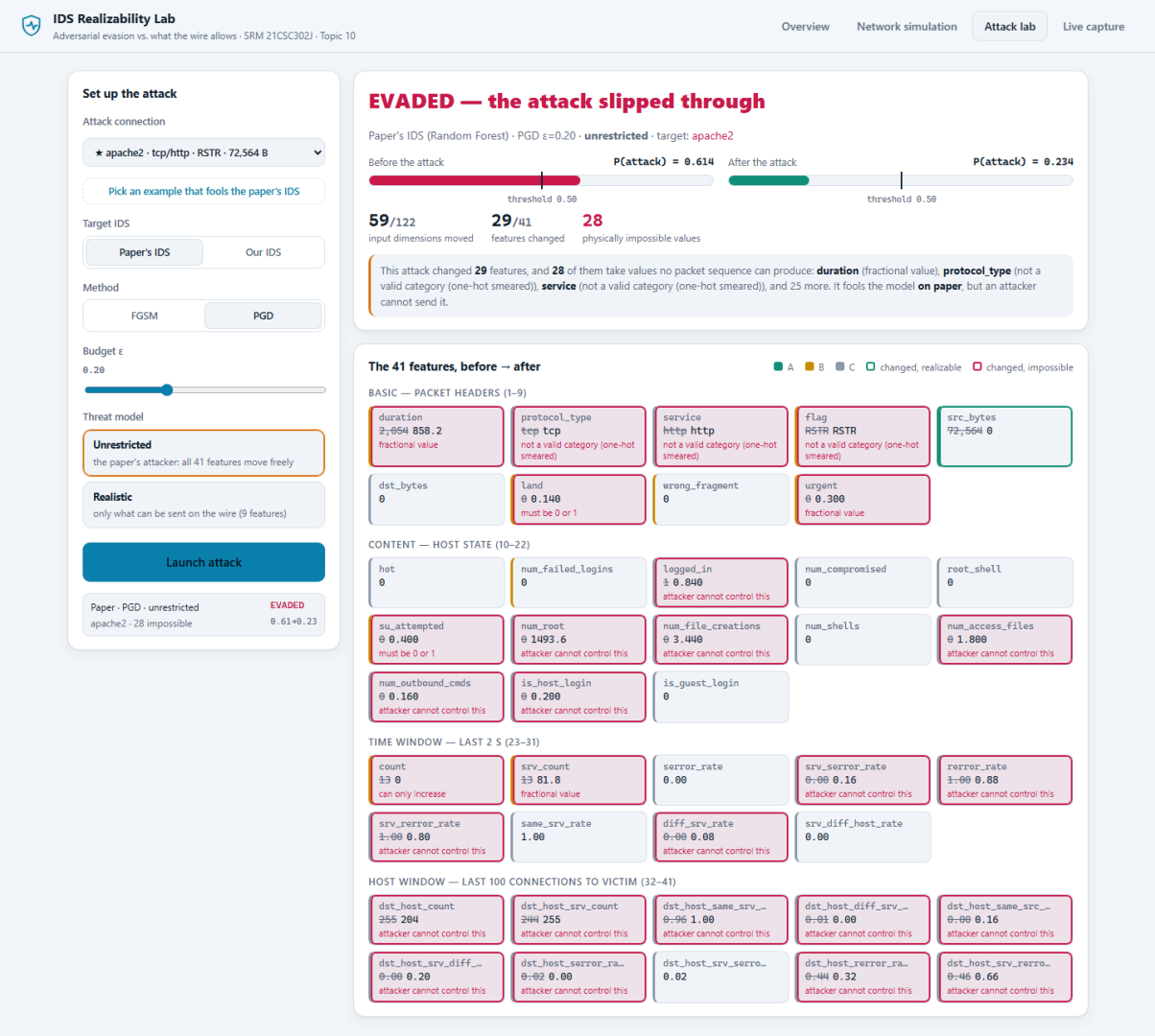


Fig 9: Attack lab: unrestricted PGD evades the paper's IDS using physically impossible values

In realistic mode (Fig 10) the 32 frozen features are locked. The attacker can still evade the paper's IDS here, by setting duration and src_bytes to zero. This illustrates the remaining limitation discussed in Section 7.2: those values are legal on the wire, but an apache2 flood that sends no bytes is no longer an attack.

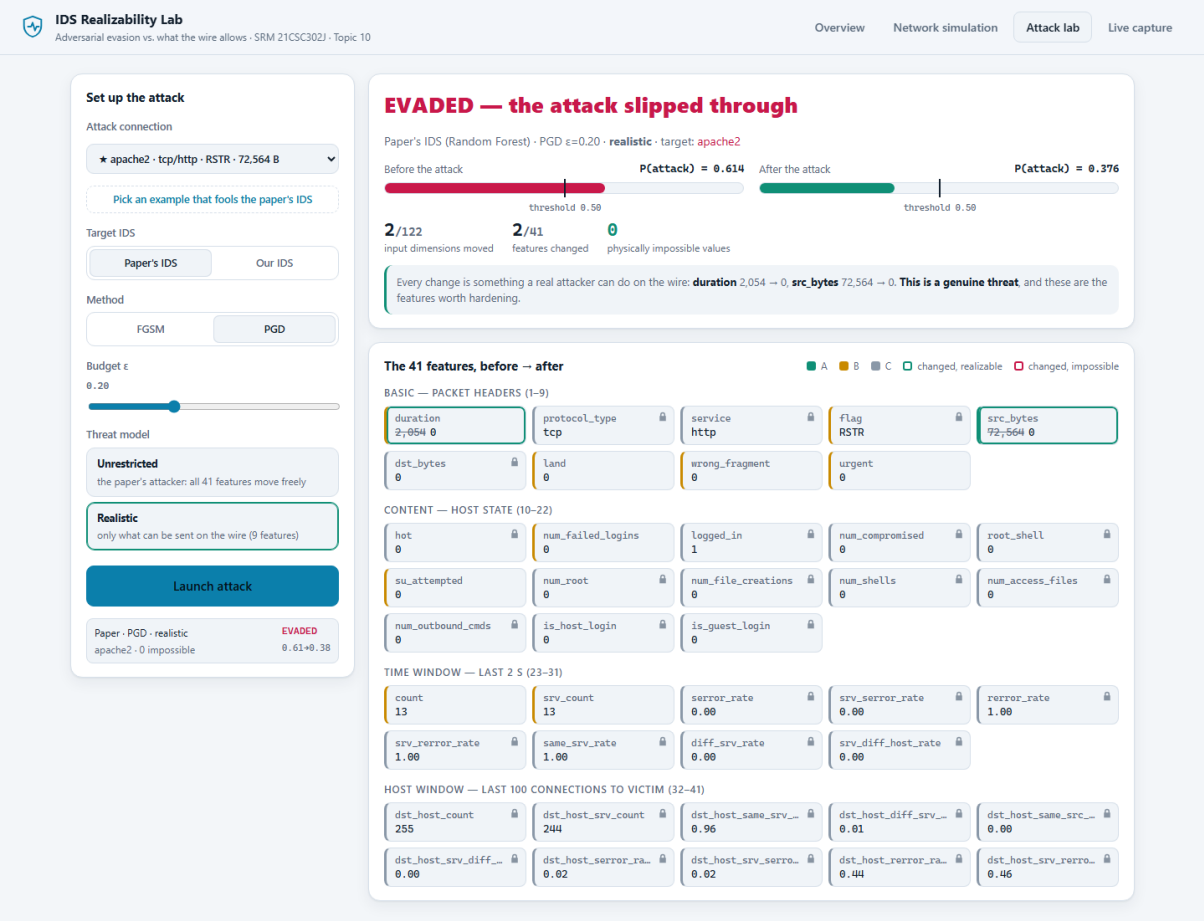


Fig 10: Attack lab: a realistic attack on the same connection; frozen features are locked

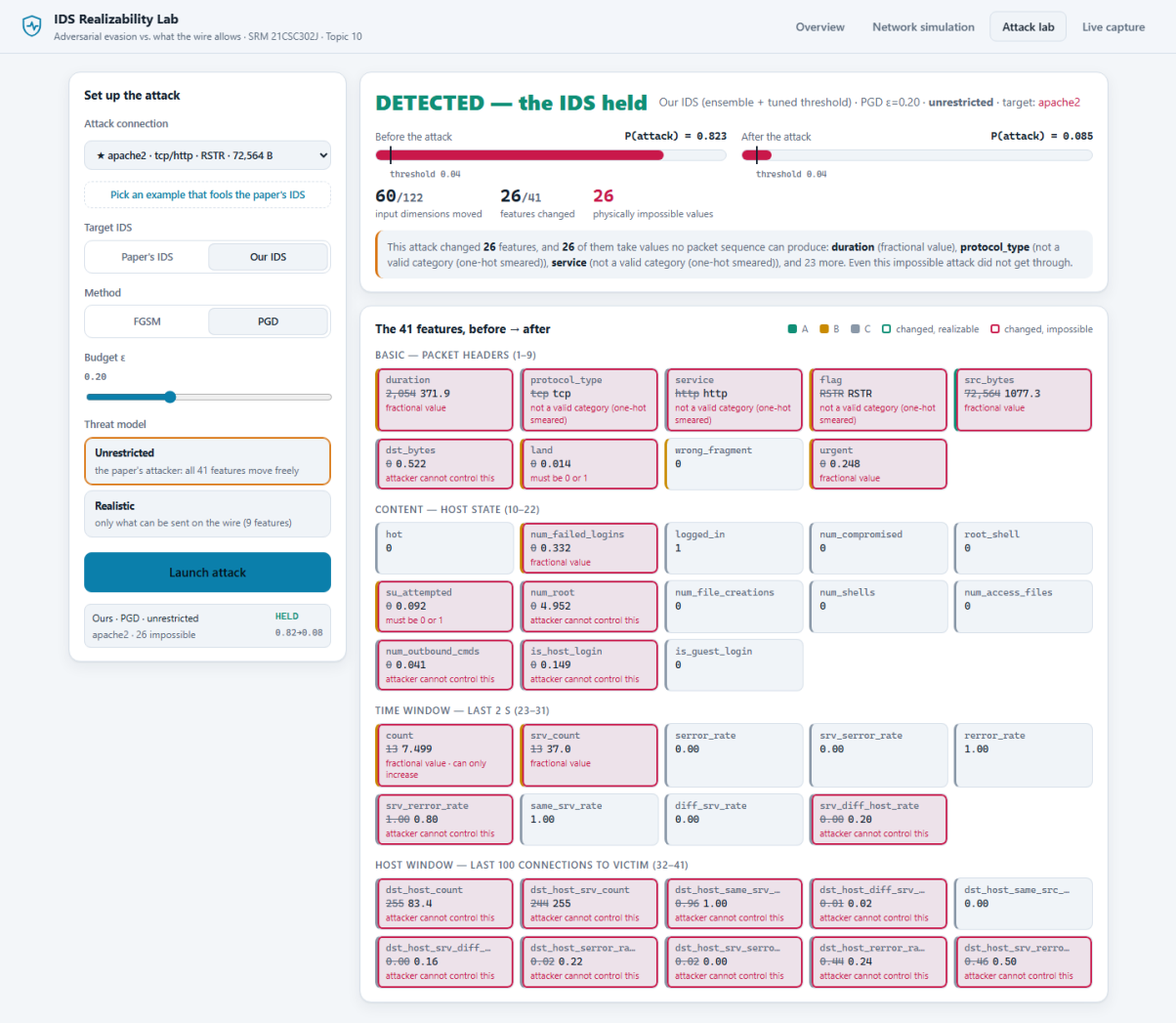


Fig 11: Attack lab: the same unrestricted attack against our IDS is still detected

7. Discussion

7.1 Comparison with Related Work

Constraining adversarial attacks on intrusion detectors is an established research direction. Table 14 summarises the closest work and how this study differs.

Table 14: Related work

Work	What it showed	Difference in this study
Teuffenbach et al. [11]	Attacks on NIDSs weighted by how hard each feature is to manipulate	Hard per-feature constraints in raw units, from a TCP/IP taxonomy
Sheatsley et al. [12]	Domain constraints do not make NIDSs robust; as few as five features can suffice	Measures the effect with an evasion metric and compares defences
Merzouk et al. [13]	Many published adversarial examples for intrusion detection are invalid	Flags each impossible value per feature, live, in the demonstration
Apruzzese et al. [14]	Realistic attacker capabilities against ML-based NIDSs	Applies a realistic threat model to the specific published pipeline
Simonetto et al. [15]	Constrained attacks and constrained adversarial retraining	Shows a calibrated threshold beats unconstrained adversarial training
Pierazzi et al. [16]	Problem-space versus feature-space attacks in general	Grounds problem-space limits in TCP/IP connection semantics

The specific contributions of this study are therefore not the idea of constrained attacks itself, but (i) showing that whole-test-set accuracy under attack conflates evasion with benign false alarms, (ii) showing that a calibrated threshold outperforms adversarial training under realistic constraints, and (iii) a pipeline that checks feature controllability on real packet captures.

7.2 Limitations

- **Dataset age:** NSL-KDD derives from 1999 traffic. The conclusions should be confirmed on a modern dataset such as UNSW-NB15 or CIC-IDS2017.
- **Single runs:** each experiment uses one fixed seed. Only the threshold selection was repeated over several splits.
- **Hand-assigned classes:** the realizability classes are a threat-model judgement. The live capture lab is designed to test them, but has not yet been run.
- **Functional realizability:** the mask enforces what the wire allows, not what keeps an attack working. Setting a flood's payload to zero is legal but defeats the flood, so the realistic evasion rates reported here are upper bounds.
- **Transfer attacks only:** stronger query-based attacks against the deployed model were not evaluated.
- **Content features in captures:** features such as logged_in describe host state and are not visible on the wire, so they are zero in captured traffic.

8. Conclusion and Future Work

An adversarial example is only a threat if a real attacker can send it. Classifying the 41 NSL-KDD features by who controls them over TCP/IP shows that an attacker can move only 9 of them, and enforcing that classification inside FGSM and PGD substantially weakens the published attacks. Measuring the right quantity matters as much: the conventional accuracy-under-attack protocol mostly records false alarms on benign traffic, while the evasion rate isolates attacks that actually get through.

An IDS built around a threshold calibrated on held-out attack types raised accuracy on unseen attacks from 77.9% to 89.6% and F1 from 0.765 to 0.907, and reduced unrestricted evasion from 31.1% to 4.9%. Against realistic attackers it was more robust than adversarial training, which had been fitted to perturbations that cannot occur.

Future work:

- Run the capture lab on real traffic and add the measured realizability results to Section 6.6.
- Repeat the experiments on a modern dataset (UNSW-NB15, CIC-IDS2017) and over several seeds.
- Add functional constraints so that attacks must also remain effective, not only realizable.
- Train the defence on realistic perturbations instead of unrestricted ones.

References

[1] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA), 2009, pp. 1–6.

[2] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[3] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” in Proc. ICLR, 2018.

[4] A. Kurakin, I. Goodfellow, and S. Bengio, “Adversarial machine learning at scale,” in Proc. ICLR, 2017.

[5] M.-I. Nicolae et al., “Adversarial Robustness Toolbox v1.0.0,” arXiv:1807.01069, 2018.

[6] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[7] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[8] P. Geurts, D. Ernst, and L. Wehenkel, “Extremely randomized trees,” Machine Learning, vol. 63, no. 1, pp. 3–42, 2006.

[9] W. Lee and S. J. Stolfo, “A framework for constructing features and models for intrusion detection systems,” ACM Trans. Information and System Security, vol. 3, no. 4, pp. 227–261, 2000.

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[11] M. Teuffenbach, E. Piatkowska, and P. Smith, “Subverting network intrusion detection: Crafting adversarial examples accounting for domain-specific constraints,” in Machine Learning and Knowledge Extraction (CD-MAKE 2020), Springer LNCS, 2020.

[12] R. Sheatsley et al., “Adversarial examples in constrained domains,” arXiv:2011.01183, 2020; Journal of Computer Security, IOS Press.

[13] M. A. Merzouk, F. Cuppens, N. Boulahia-Cuppens, and R. Yaich, “A deeper analysis of adversarial examples in intrusion detection,” in Risks and Security of Internet and Systems (CRiSIS 2020), Springer, 2021.

[14] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, and M. Colajanni, “Modeling realistic adversarial attacks against network intrusion detection systems,” Digital Threats: Research and Practice, vol. 3, no. 3, art. 31, 2022.

[15] T. Simonetto et al., “A unified framework for adversarial attack and defense in constrained feature space,” in Proc. Int. Joint Conf. Artificial Intelligence (IJCAI), 2022.

[16] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, “Intriguing properties of adversarial ML attacks in the problem space,” in Proc. IEEE Symp. Security and Privacy, 2020.

[17] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Pearson, 2021.

[18] [Authors], "[Title of your ICADS 2026 paper]," in Proc. [full ICADS 2026 conference name], 2026, pp. [xx-yy].

[19] W. R. Stevens, TCP/IP Illustrated, Volume 1: The Protocols. Addison-Wesley, 1994.

[20] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in Proc. IEEE Symp. Security and Privacy, 2010, pp. 305–316.

Appendix A: Full Feature Realizability Taxonomy

B* marks flag: Class B, but frozen in the continuous attacks because it is categorical.

Table 15: The 41 NSL-KDD features and who controls them

#	Feature	Group	Class	Controlled by / constraint
1	duration	Basic	B	Attacker holds the connection; whole seconds, ≥ 0
2	protocol_type	Basic	C	Choice made when the attack is designed; changing it is a different attack
3	service	Basic	C	Target port chosen at design time; categorical
4	flag	Basic	B*	Reflects how the victim answered; categorical, frozen in the attack
5	src_bytes	Basic	A	Bytes the attacker sends; whole bytes, ≥ 0
6	dst_bytes	Basic	C	Bytes the victim sends back
7	land	Basic	B	Source = destination spoofing; binary and self-incriminating
8	wrong_fragment	Basic	B	Malformed fragments; small integer, self-incriminating
9	urgent	Basic	B	Packets with URG set; small integer
10	hot	Content	C	Emerges from session content on the host
11	num_failed_logins	Content	B	Attacker chooses how many tries; can only increase
12	logged_in	Content	C	1 only if authentication actually succeeded
13	num_compromised	Content	C	Victim state after compromise
14	root_shell	Content	C	1 only if a root shell was obtained
15	su_attempted	Content	B	Attacker action; binary
16	num_root	Content	C	Victim state; success-dependent
17	num_file_creations	Content	C	Victim state after compromise
18	num_shells	Content	C	Victim state after compromise
19	num_access_files	Content	C	Victim state after compromise
20	num_outbound_cmds	Content	C	Constant 0 in the whole dataset
21	is_host_login	Content	C	Depends on the account used
22	is_guest_login	Content	C	Depends on the account used
23	count	Time (2 s)	B	Attacker adds connections; can only increase
24	srv_count	Time (2 s)	B	Same, per service; can only increase
25	serror_rate	Time (2 s)	C	Victim's SYN errors over the window
26	srv_serror_rate	Time (2 s)	C	Same, per service
27	rerror_rate	Time (2 s)	C	Victim's rejections over the window
28	srv_rerror_rate	Time (2 s)	C	Same, per service
29	same_srv_rate	Time (2 s)	C	Ratio over all recent connections
30	diff_srv_rate	Time (2 s)	C	Ratio over all recent connections
31	srv_diff_host_rate	Time (2 s)	C	Ratio over all recent connections
32	dst_host_count	Host (100)	C	Last 100 connections to the victim, from anyone
33	dst_host_srv_count	Host (100)	C	Same, per service
34	dst_host_same_srv_rate	Host (100)	C	Ratio over third-party history
35	dst_host_diff_srv_rate	Host (100)	C	Ratio over third-party history
36	dst_host_same_src_port_rate	Host (100)	C	Ratio over third-party history
37	dst_host_srv_diff_host_rate	Host (100)	C	Ratio over third-party history
38	dst_host_serror_rate	Host (100)	C	Victim errors over third-party history
39	dst_host_srv_serror_rate	Host (100)	C	Victim errors over third-party history
40	dst_host_rerror_rate	Host (100)	C	Victim rejections over third-party history
41	dst_host_srv_rerror_rate	Host (100)	C	Victim rejections over third-party history

Appendix B: Reproducing the Results

From the project folder, with the Python environment set up as in README.md and the NSL-KDD files in data/:

```
.venv/Scripts/python.exe src/ids_model.py        # build the improved IDS
.venv/Scripts/python.exe src/experiment.py       # attacks, mask, defence, evasion
.venv/Scripts/python.exe src/benchmark.py --robust   # threshold robustness
.venv/Scripts/python.exe src/shap_analysis.py    # feature importance
.venv/Scripts/python.exe src/plots.py            # figures
.venv/Scripts/python.exe -m pytest tests/        # test suite
.venv/Scripts/python.exe demo/app.py             # demonstration application
.venv/Scripts/python.exe report/build_report.py  # this report
```

To run the capture lab on a machine with Docker:

```
cd lab
bash run_capture.sh full
# then drop lab/pcaps/capture.pcap into the demo's Live capture view
```